

CSE144 Final Project Report (S26)

David Guerrero-Pantoja	dguerr15@ucsc.edu	1957759
Ivan Severinov	ivseveri@ucsc.edu	1982578
Tanvi Herwadkar	therwadk@ucsc.edu	1985778

1. Introduction

The goal of this project was to develop an image classification model for a dataset where all of the training images are classified in 100 classes. Since there are only 10 images per class, transfer learning is an appropriate learning method because pretrained CNNs already contain information learned from large-scale datasets. Instead of developing a model from scratch, transfer learning allows us to fine tune a pre-existing model which results in efficiency.

In our approach we used a pretrained EfficientNet-V2-S model from TorchVision. Our code takes a two phased approach where the first phase trains the classifier head only and the second phase fine tunes the whole model. Our final model achieved approximately 76% accuracy.

2. Dataset

The data we used to create our model consisted of 1000 training images which were split up into 100 labeled classes with 10 images in each class. Each subdirectory in the training folder was labeled 0-99 and acted as the label for that class. There was also a test set of 1000 images that were unlabeled.

Before training we resized all the images to 224x224 pixels (the format expected by EfficientNet). The normalization was done using the ImageNet normalization stats. In terms of data augmentation we applied the following: random horizontal flip, random rotation, random resized crop, color jitter.

3. Implementation

3.1 Model

Our model is EfficientNet-V2-S from TorchVision. This model came recommended because it provides a balance between accuracy and efficiency. Also we read that it is supposed to work well even when training data is limited. The original classifier layer was designed for 1000 ImageNet classes, which we replaced with a fully-connected layer of our own that outputted the 100 classes that we were interested in. The first phase of the code was retraining the model. We did this by freezing the pretrained weights and only changing the classifier head. Then in the second phase we fine tuned on the full network.

3.2 Training

The model was trained using the cross-entropy loss function.

Optimizer: AdamW

Learning Rate:

- Phase 1: $8e-4$ for the classifier head
- Phase 2: $6e-5$ for the pretrained backbone and $3e-4$ for the classifier head

Batch Size: 16

Epochs:

- Phase 1: 6 epochs
- Phase 2: up to 50 epochs with early stopping patience of 12

Weight Decay: $5e-4$

A learning rate scheduler was used during training to improve convergence.

The model was trained using PyTorch and TorchVision.

Software:

Python 3.12.1

PyTorch ≥ 2.0

TorchVision ≥ 0.15

4. Experiments

Our baseline experiment used the pretrained EfficientNet-V2-S model with standard resizing and normalization.

We experimented with several training and regularization strategies, including:

- Different learning rates
- Data augmentation methods
- Validation split configurations
- Fine-tuning strategies
- Weight decay values

Validation accuracy was monitored throughout training to determine the best-performing model checkpoint.

Additional experiments included testing the effect of stronger augmentations and comparing frozen-feature extraction versus full-network fine-tuning.

5. Results

The submitted model checkpoint, `best_model_On_Kaggle.pt`, achieved approximately 71% validation accuracy locally and was used to generate the final `submission.csv` file. Inference used 5-view test-time augmentation, where predictions from multiple transformed views were softmax-averaged before selecting the final class with `argmax`.

The repository also contained a separate checkpoint, `best_model.pt`, which achieved 77.36% validation accuracy at epoch 16. However, this checkpoint was not the one used for the submitted Kaggle file.

6. Discussion

Transfer learning proved highly effective for this project due to the limited amount of labeled training data. The pretrained EfficientNet backbone already contained useful image representations, allowing the model to converge quickly and achieve strong accuracy.

Data augmentation helped reduce overfitting and improved generalization performance. Fine-tuning the full network produced better results than only training the classifier head.

One limitation of the project was the extremely small dataset size. Some classes contained only a few distinct visual examples, making generalization difficult.

Future improvements could include:

- Testing Vision Transformer (ViT) architectures
- Using larger pretrained models
- Applying stronger regularization

7. Reproducibility

To make the experiment reproducible, the project was implemented in a single Jupyter notebook, `CSE-144_Final_Project.ipynb`. The notebook contains the full pipeline for loading the dataset, creating the train/validation split, applying preprocessing, training the EfficientNet-V2-S model, saving checkpoints, and generating the final file.

The experiment used Python 3.12.1, PyTorch ≥ 2.0 , and TorchVision ≥ 0.15 . The model was trained using TorchVision's EfficientNet-V2-S architecture with `EfficientNet_V2_S_Weights.IMAGENET1K_V1` pretrained weights.

Training was divided into two phases. Phase 1 froze the pretrained backbone and trained only the classifier head for 6 epochs. Phase 2 unfroze the full model and fine-tuned it for up to 50 epochs with early stopping patience of 12. The optimizer was AdamW with weight decay $5e-4$ and batch size 16. The final submission was generated by loading `best_model_On_Kaggle.pt` and applying 5-view test-time augmentation before writing predictions to `submission.csv`.

8. Team Contributions

David: Hyperparameter tuning and training experiments

Tanvi: Research on CNN architectures and report preparation

Ivan: Research on Vision Transformers and model comparisons

All members: Model implementation, debugging, and evaluation

9. References

Paszke, A., Gross, S., Massa, F., et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.

TorchVision Documentation. Pretrained Models and Weights.

Tan, M., & Le, Q. V. (2021). *EfficientNetV2: Smaller Models and Faster Training*. International Conference on Machine Learning (ICML).

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). *ImageNet: A Large-Scale Hierarchical Image Database*. CVPR.